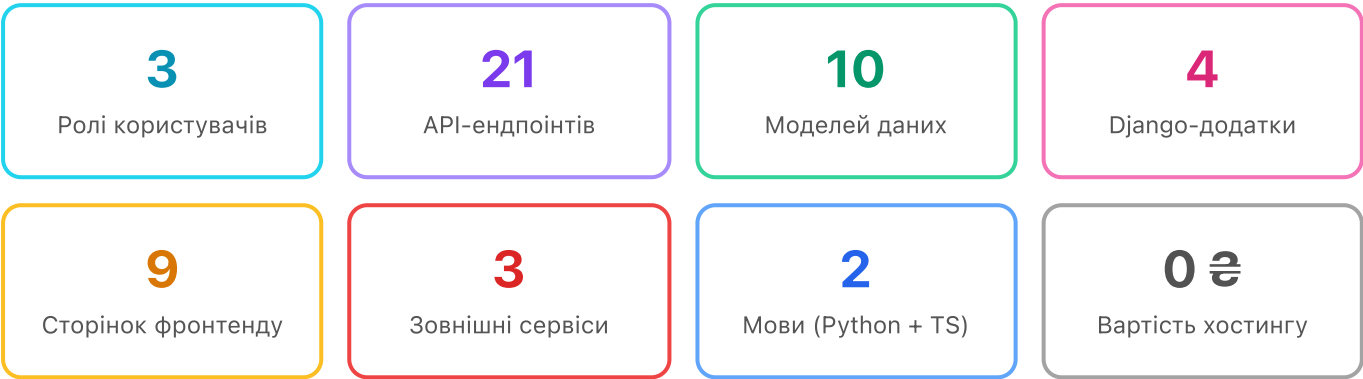


NexLearn — Повна структура проекту

Проект в числах



Загальна ідея

NexLearn — **веб-орієнтована інформаційна система супроводу навчального процесу** для центрів підвищення кваліфікації. Система складається з **трьох частин**, які працюють разом:

- 1. **Frontend** — Next.js-додаток на `nexlearn.space`, де викладачі створюють курси і тести, а слухачі вчаться і складають іспити
- 2. **Backend API** — Django REST Framework сервер, який зберігає всі дані, перевіряє права і генерує сертифікати
- 3. **База даних PostgreSQL** — у хмарі Supabase, зберігає користувачів, курси, тести, результати

Усі три частини спілкуються між собою через **REST API** з JWT-автентифікацією.

Де що живе

Frontend (Vercel)	Backend (Railway / Render)
nexlearn.space src/ app/ page.tsx ← головна login/ ← вхід register/ ← реєстрація dashboard/ courses/ ← курси tests/ ← тести certificates/ profile/ ← профіль users/ ← адмін	Django + DRF config/ settings.py ← налаштування urls.py ← маршрути wsgi.py ← gunicorn apps/ users/ models.py ← акаунти, JWT views.py ← User з ролями urls.py courses/ ← курси models.py ← Course, Material

└─ components/		└─ views.py	← + comments, rate
└─ ui/	← кнопки	└─ tests_module/	← тестування
└─ layout/	← header	└─ models.py	← Test, Question
└─ lib/		└─ views.py	← + submit, results
└─ api.ts	← axios	└─ certificates/	← сертифікати
└─ auth.ts	← JWT	└─ models.py	← UUID-номери
└─ types/	← TS-типи	└─ views.py	
└─ public/		└─ manage.py	
└─ next.config.ts	← CSP	└─ requirements.txt	
└─ vercel.json	← деплой	└─ Procfile	← gunicorn
		└─ build.sh	← міграції + admin

Database (Supabase)

PostgreSQL

users_user	← кастомна модель з role
courses_course	← курси
courses_coursematerial	← файли курсу
courses_courseenrollment	← записи
courses_coursecomment	← коментарі
courses_courserating	← оцінки
tests_module_test	← тести
tests_module_testquestion	← запитання
tests_module_testresult	← результати
certificates_certificate	← сертифікати з UUID

Як працює фронтенд (nexlearn.space)

Стек технологій

- **Next.js 16** — React-фреймворк з App Router (нова система маршрутизації)
- **React 19** — UI-бібліотека
- **TypeScript** — типізація замість «голоного» JavaScript
- **Tailwind CSS 4** — стилі через CSS-класи
- **Axios** — HTTP-клієнт для запитів до Django API
- **jsPDF** — генерація PDF-сертифікатів у браузері
- **Lucide React** — набір SVG-іконок

Відкриття сторінки

1. Користувач заходить на `nexlearn.space`
2. Vercel CDN віддає згенеровану Next.js сторінку (SSR / SSG)
3. Browser завантажує React-бандл і гідрує сторінку
4. `api.ts` створює axios-клієнт з base URL = `NEXT_PUBLIC_API_URL`
5. Якщо в `localStorage` є `access_token` — він додається до заголовка Authorization

6. Якщо токен прострочений (401) — автоматичний refresh через `/api/auth/refresh/`

Сторінки фронтенду

URL	Доступ	Що робить
<code>/</code>	Усі	Лендінг — опис системи, кнопки «Вхід» і «Реєстрація»
<code>/login</code>	Усі	Форма входу: username + пароль → JWT
<code>/register</code>	Усі	Реєстрація з вибором ролі (тільки teacher / student)
<code>/dashboard</code>	Авторизовані	Особистий кабінет: статистика, останні курси, тести
<code>/dashboard/courses</code>	Авторизовані	Каталог курсів: пошук, фільтри, рейтинги
<code>/dashboard/courses/[id]</code>	Авторизовані	Деталі курсу: матеріали, коментарі, кнопка «Записатися»
<code>/dashboard/tests</code>	Авторизовані	Список тестів за курсами
<code>/dashboard/tests/[id]</code>	Авторизовані	Пройдення тесту: запитання + варіанти + таймер
<code>/dashboard/certificates</code>	Авторизовані	Мої сертифікати з можливістю завантажити PDF
<code>/dashboard/profile</code>	Авторизовані	Редагування профілю: ім'я, телефон, аватар, біо
<code>/dashboard/users</code>	Тільки адмін	Управління користувачами: змінити роль

Як фронтенд робить запити до бекенду

```
// src/lib/api.ts – axios-клієнт з інтерсепторами

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('access_token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

api.interceptors.response.use(
  (response) => response,
  async (error) => {
    // 401? → автоматичний refresh
    if (error.response?.status === 401 && !originalRequest._retry) {
      const refresh = localStorage.getItem('refresh_token');
      const { data } = await axios.post('/api/auth/refresh/', { refresh });
      localStorage.setItem('access_token', data.access);
      // Повторюємо оригінальний запит з новим токеном
      return api(originalRequest);
    }
    return Promise.reject(error);
  }
);
```

Зберігання токенів

- **access_token** — короткий (60 хв), зберігається в `localStorage`
- **refresh_token** — довгий (7 днів), теж в `localStorage`, з ротацією
- При логауті — обидва видаляються
- При rotation — старий refresh додається в `token_blacklist` (Django)

Безпеківі заголовки фронтенду (next.config.ts)

Заголовок	Що робить
<code>X-Content-Type-Options: nosniff</code>	Не дозволяє браузеру «вгадувати» тип файлу
<code>X-Frame-Options: DENY</code>	Заборона вбудовування в iframe (захист від clickjacking)
<code>Content-Security-Policy</code>	Дозволяє завантажувати скрипти/стили/картинки тільки з нашого API
<code>Strict-Transport-Security</code>	Браузер завжди використовує HTTPS (HSTS, 1 рік)
<code>Referrer-Policy</code>	Не передає повний URL чужим сайтам
<code>Permissions-Policy</code>	Заборона доступу до камери, мікрофона, геолокації

Як працює бекенд (Django + DRF)

Стек технологій

- **Python 3.12** — мова
- **Django 6** — веб-фреймворк
- **Django REST Framework 3.15** — REST API
- **django-rest-framework-simplejwt 5.4** — JWT-автентифікація
- **django-cors-headers** — CORS
- **django-ratelimit** — обмеження запитів
- **psycopg 3.3** — драйвер PostgreSQL
- **Pillow** — обробка картинок (аватари)
- **WhiteNoise** — віддача статички через gunicorn
- **gunicorn** — WSGI-сервер

Структура додатків (Django apps)

```
apps/  
├── users/           ← користувачі і автентифікація  
│   ├── models.py   ← User (AbstractUser + role + phone + bio + avatar)  
│   └── serializers.py ← Register, User, ChangeRole
```

└─ views.py	← Register, Me, UserList, ChangeRole
└─ urls.py	← /api/auth/* (з rate-limit)
└─ urls_admin.py	← /api/users/* (тільки адмін)
└─ courses/	← курси і пов'язане
└─ models.py	← Course, CourseMaterial, CourseEnrollment, CourseComment, CourseRating
└─ serializers.py	← List, Detail, Create, Material, Comment, Rating
└─ views.py	← CRUD + enroll + progress + rate
└─ urls.py	← /api/courses/*
└─ tests_module/	← тести
└─ models.py	← Test, TestQuestion, TestResult
└─ serializers.py	← з + без correct_answer (для студентів)
└─ views.py	← Submit (підрахунок балів + видача сертифіката)
└─ urls.py	← /api/tests/*
└─ certificates/	← сертифікати
└─ models.py	← Certificate з UUID-номером
└─ serializers.py	
└─ views.py	← List, Detail (тільки чужі бачать адміни)
└─ urls.py	← /api/certificates/*

Модель User

```
class User(AbstractUser):
    class Role(TextChoices):
        ADMIN = 'admin'          # повний доступ
        TEACHER = 'teacher'      # створює курси і тести
        STUDENT = 'student'      # вчиться і складає тести

    role = CharField(choices=Role.choices, default=STUDENT)
    phone = CharField(max_length=20, blank=True)
    bio = TextField(blank=True)
    avatar = ImageField(upload_to='avatars/', validators=[file<=2MB])

    @property
    def is_admin(self): return self.role == 'admin'
    @property
    def is_teacher(self): return self.role == 'teacher'
    @property
    def is_student(self): return self.role == 'student'
```

Зв'язки моделей

```
User —one-to-many—> Course (через teacher)
User —many-to-many—> Course (через CourseEnrollment)
User —many-to-many—> Course (через CourseRating: 1-5 ★)
User —one-to-many—> CourseComment

Course —one-to-many—> CourseMaterial (файли до 10MB)
Course —one-to-many—> Test
```

Course —one-to-many—> Certificate

Test —one-to-many—> TestQuestion (з options + correct_answer)

Test —one-to-many—> TestResult (бал + passed)

Certificate (UUID-номер) —unique—> (student, course)

Ролі і права

Дія	 Адмін	 Викладач	 Слухач
Реєстрація	Тільки через Django admin	✓ через сайт	✓ через сайт
Курси	Бачить усі (включаючи чернетки), редагує будь-який	Створює свої, редагує тільки свої, бачить чернетки	Бачить тільки опубліковані
Матеріали курсу	Завантажує до будь-якого курсу	Тільки до своїх курсів	Завантажує (читає) — після запису на курс
Тести	Створює і редагує всі	Створює тести до своїх курсів	Складає тести курсів, на які записаний
Спроби тесту	Без обмежень	Без обмежень	Максимум 3 спроби на тест
Результати тестів	Бачить усі	Бачить результати своїх тестів	Бачить тільки свої результати
Експорт результатів	✓ CSV з усіма даними	✓ CSV для своїх тестів	×
Сертифікати	Бачить усі видані	Бачить видані за свої курси	Бачить тільки свої
Зміна ролей	✓ через <code>/dashboard/users</code>	×	×
Коментарі і оцінки	Будь-який курс	Будь-який курс	Будь-який курс

Як перевіряється роль

```
# Backend – DRF Permission classes
class IsTeacherOrAdmin(BasePermission):
    def has_permission(self, request, view):
        return request.user.role in ('teacher', 'admin')

class IsOwnerOrAdmin(BasePermission):
    def has_object_permission(self, request, view, obj):
        return obj.teacher == request.user or request.user.is_admin

class IsAdmin(BasePermission):
    def has_permission(self, request, view):
        return request.user.is_authenticated and request.user.is_admin
```

API-ендпоінти

Автентифікація (/api/auth/)

Ендпоінт	Метод	Доступ	Що робить
/api/auth/register/	POST	Усі (3/хв per IP)	Реєстрація: username, email, password, role (teacher/student)
/api/auth/login/	POST	Усі (5/хв per IP)	Вхід: username + password → access_token + refresh_token
/api/auth/refresh/	POST	Усі	Оновити access за допомогою refresh (з ротацією)
/api/auth/me/	GET	Авторизовані	Дані поточного користувача
/api/auth/me/	PATCH	Авторизовані	Редагувати профіль (ім'я, телефон, біо, аватар)

Користувачі (/api/users/) — тільки адмін

Ендпоінт	Метод	Що робить
/api/users/	GET	Список усіх користувачів з пагінацією
/api/users/{id}/role/	PATCH	Змінити роль (admin / teacher / student)

Курси (/api/courses/)

Ендпоінт	Метод	Доступ	Що робить
/api/courses/	GET	Усі	Список курсів (студенти бачать тільки published)
/api/courses/	POST	Викладач/Адмін	Створити курс (статус draft за замовчуванням)
/api/courses/{id}/	GET	Усі	Деталі курсу + матеріали + рейтинг
/api/courses/{id}/	PUT/PATCH	Власник/Адмін	Редагувати курс
/api/courses/{id}/	DELETE	Власник/Адмін	Видалити курс
/api/courses/{id}/enroll/	POST	Авторизовані	Записатися на курс
/api/courses/{id}/materials/	GET	Авторизовані	Список матеріалів
/api/courses/{id}/materials/	POST	Викладач (свій)	Завантажити файл (≤10MB)
/api/courses/{id}/progress/	GET	Авторизовані	% пройдених тестів + чи є сертифікат
/api/courses/{id}/comments/	GET	Авторизовані	Список коментарів

<code>/api/courses/{id}/comments/</code>	POST	Авторизовані	Додати коментар
<code>/api/courses/{id}/rate/</code>	GET	Авторизовані	Моя оцінка курсу
<code>/api/courses/{id}/rate/</code>	POST	Авторизовані	Поставити оцінку 1-5 ★

Тести (`/api/tests/`)

Ендпоінт	Метод	Доступ	Що робить
<code>/api/tests/?course={id}</code>	GET	Авторизовані	Список тестів за курсом
<code>/api/tests/</code>	POST	Викладач/ Адмін	Створити тест із запитаннями
<code>/api/tests/{id}/</code>	GET	Авторизовані	Деталі тесту (студенти не бачать <code>correct_answer</code>)
<code>/api/tests/{id}/submit/</code>	POST	Записані на курс	Здати тест: <code>{"answers": [0, 2, 1]}</code> → бал + сертифікат
<code>/api/tests/{id}/results/</code>	GET	Авторизовані	Список результатів тесту
<code>/api/tests/{id}/export/</code>	GET	Викладач/ Адмін	Експорт результатів у CSV
<code>/api/tests/my-results/</code>	GET	Авторизовані	Підсумок: найкращий бал + кількість спроб для кожного тесту

Сертифікати (`/api/certificates/`)

Ендпоінт	Метод	Доступ	Що робить
<code>/api/certificates/</code>	GET	Авторизовані	Список (студенти бачать тільки свої)
<code>/api/certificates/{id}/</code>	GET	Авторизовані	Деталі сертифіката для рендерингу PDF

Тестування — як працює

Створення тесту викладачем

```
POST /api/tests/
{
  "title": "Тест 1: Основи Python",
  "description": "Перевірка базових знань",
  "course": 5,
  "passing_score": 70,
  "time_limit": 30,           // хвилини, 0 = без обмежень
  "questions": [
    {
      "question_text": "Що take list comprehension?",
```



```

    "options": ["Цикл", "Скорочений запис створення списку",
               "Сортування", "Фільтрація"],
    "correct_answer": 1,    // індекс правильної відповіді
    "order": 0
  },
  ...
]
}

```

Проходження тесту слухачем

1. Слухач відкриває `/dashboard/tests/{id}`
2. Frontend робить `GET /api/tests/{id}/` — без поля `correct_answer`
3. Слухач відповідає, фронтенд накопичує масив індексів: `[1, 0, 2, 1, ...]`
4. `POST /api/tests/{id}/submit/` з масивом
5. Backend перевіряє:
 - чи слухач записаний на курс (інакше 403)
 - чи не вичерпав 3 спроби (інакше 400)
 - довжина масиву = кількість запитань
6. Підрахунок: `score = (правильних / всього) × 100`
7. Якщо `score >= passing_score` → `passed = true`
8. Якщо `passed` і ще немає сертифіката → створює `Certificate` з UUID-номером
9. Зберігає `TestResult` з масивом відповідей (для статистики)
10. Повертає результат → фронтенд показує бал + повідомлення

Антишахрайство

- **Студенти не бачать `correct_answer`** — окремий `TestStudentDetailSerializer`
- **Перевірка `enrollment`** — не записаний на курс → не може здати тест
- **Ліміт 3 спроби** — після цього 400 Bad Request
- **Перевірка кількості відповідей** — неможливо здати «половину» тесту
- **JWT з коротким TTL** — 60 хвилин для access (гарантовано не застаріє під час тесту)

Сертифікати — як працює

Видача сертифіката

Сертифікат видається **автоматично** при першому успішному проходженні тесту курсу:

```

# tests_module/views.py — SubmitTestView
if passed:

```

```
Certificate.objects.get_or_create(
    student=request.user,
    course=test.course,
)
```

`get_or_create` = якщо вже є сертифікат — нічого не робить (через `unique_together`).

Структура сертифіката

```
Certificate
├── student (FK → User)
├── course (FK → Course)
├── certificate_number (UUID, autogen)
└── issued_at (datetime)

unique_together: (student, course)
ordering: -issued_at (свіжі вгори)
```

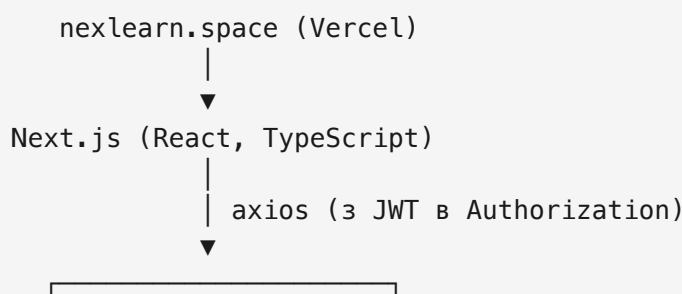
Генерація PDF на фронтенді

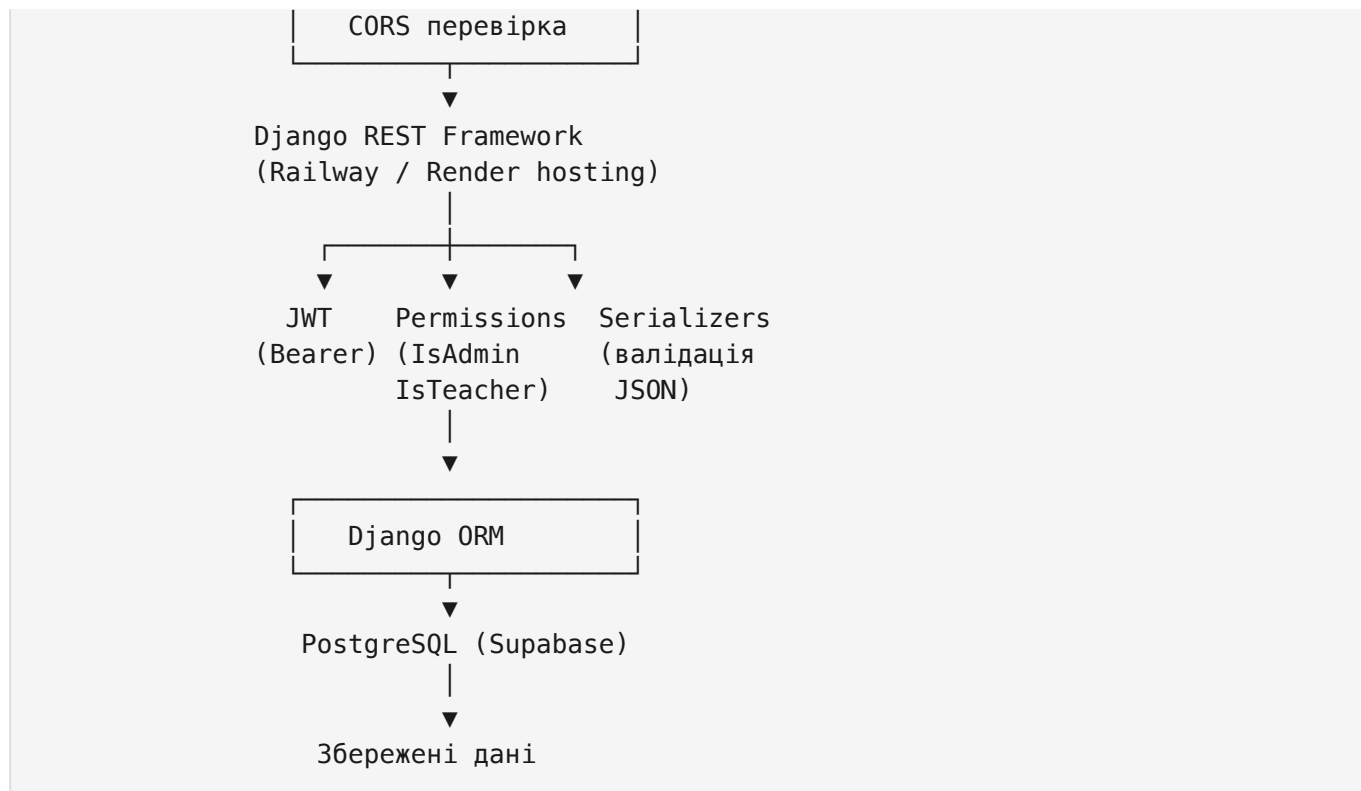
PDF генерується в **браузері** через `jsPDF`:

1. Слухач натискає «Завантажити PDF» на `/dashboard/certificates`
2. `jsPDF` створює A4-сторінку
3. Малює рамку, логотип, текст:
 - Видано: `{first_name} {last_name}`
 - За курс: `{course.title}`
 - Викладач: `{teacher.get_full_name}`
 - Номер: `{certificate_number}` (UUID)
 - Дата: `{issued_at}`
4. Зберігає як `certificate-{uuid}.pdf` на пристрій

Перевага: бекенд не витрачає CPU на рендеринг PDF, не потрібен Pillow для шрифтів і т.д.

Як зв'язані фронтенд і бекенд





1. **Окремі домени:** фронт на Vercel, бек на Railway — спілкування через CORS
2. **Stateless backend:** усі запити містять JWT, сесій немає
3. **JSON-first:** усі запити і відповіді у форматі JSON
4. **Multipart для файлів:** при завантаженні аватара або матеріалу курсу

Зберігання файлів

Що зберігається

Тип	Де	Ліміт	Приклад
Аватари юзерів	<code>media/avatars/</code>	2 MB	JPG, PNG
Матеріали курсів	<code>media/materials/</code>	10 MB	PDF, DOCX, ZIP, MP4

Де реально лежать файли

- **Локальна розробка:** `backend/media/` на диску
- **Production (Railway):** ефемерний диск контейнера — при рестарті файли зникнуть
- **Рекомендація:** для продакшену перейти на **S3 / Cloudinary / Supabase Storage** через `django-storages`

Валідація розміру файлу

```
# courses/models.py
def validate_file_size_10mb(value):
    if value.size > 10 * 1024 * 1024:
        raise ValidationError('Максимальний розмір файлу – 10MB.')

class CourseMaterial(models.Model):
    file = FileField(upload_to='materials/',
                    validators=[validate_file_size_10mb])
```

Безпека

Frontend

- **HTTPS only** — Vercel автоматично
- **CSP** — заборона `frame-ancestors`, дозвіл скриптів тільки з `self`
- **HSTS** — браузер примусово використовує HTTPS на 1 рік
- **X-Frame-Options: DENY** — захист від clickjacking
- **Permissions-Policy** — заборона камери / мікрофона / геолокації
- **Token storage в localStorage** — простий обмін, але вразливо до XSS (для бойового критичного — варто перейти на httpOnly cookies)

Backend

- **JWT з ротацією**: access 60 хв, refresh 7 днів, `BLACKLIST_AFTER_ROTATION=True`
- **Rate-limit**: `login = 5/хв per IP`, `register = 3/хв per IP`
- **Валідація паролів Django**: min 8 символів, заборона популярних, заборона числових
- **Валідація реєстрації**: неможливо створити admin через `/api/auth/register/`
- **CORS allowlist**: тільки `nexlearn.space` (через `CORS_ALLOWED_ORIGINS`)
- **CSRF**: відключено для API (JWT-only), увімкнено для Django admin
- **Хешування паролів**: PBKDF2-SHA256 (Django default, сотні тисяч ітерацій)
- **Секретний URL адмінки**: `/mgmt-panel-x9k2/` замість стандартного `/admin/`
- **SQL Injection**: захищено через Django ORM (параметризовані запити)
- **Permissions per endpoint**: кожен view має свою `permission_classes`

Database (Supabase)

- **SSL connection** — обов'язково
- **Row Level Security** — Supabase підтримує (опціонально, дублює Django)
- **Бекапи** — Supabase робить автоматично (на безкоштовному плані 7 днів)

- **Connection pooling** — через `conn_max_age=600` у Django

Потоки даних: Повний цикл

Реєстрація і вхід студента

Frontend	Backend	Supabase
1. /register form POST /api/auth/register/ →	rate-limit перевірка validate password match validate role != 'admin' set_password (PBKDF2) → return UserSerializer	INSERT users_user
2. /login form POST /api/auth/login/ →	rate-limit перевірка authenticate user generate access + refresh { access, refresh }	
3. localStorage.setItem(...)		
4. Запити з заголовком Authorization: Bearer ... →	JWTAuthentication parse token, verify signature set request.user permission_classes check	
5. Якщо 401: POST /api/auth/refresh/ →	verify refresh token blacklist старий refresh → generate new pair { access, refresh }	INSERT token_blacklist

Викладач створює курс і тест

1. POST /api/courses/ { title, description, status='draft' }	→ IsTeacherOrAdmin? teacher = request.user INSERT courses_course ← { id: 5, ... }
2. POST /api/courses/5/materials/ FormData: file=lecture.pdf	→ IsOwnerOrAdmin? перевірка size <= 10MB FileField → media/materials/ INSERT courses_coursematerial ← { id, file_url, ... }
3. PATCH /api/courses/5/	→ status='published' ← оновлений курс
4. POST /api/tests/	→ IsTeacherOrAdmin?

```
{ course: 5, title, questions: [...] }    transaction:
      INSERT tests_module_test
      INSERT tests_module_testquestion (xN)
      ← { id, ... }
```

Студент проходить тест → отримує сертифікат

1. GET /api/courses/5/ → курс опубліковано? → так
return CourseDetailSerializer
2. POST /api/courses/5/enroll/ → CourseEnrollment.get_or_create
INSERT courses_courseenrollment
← { enrollment }
3. GET /api/tests/?course=5 → список тестів курсу
4. GET /api/tests/3/ → TestStudentDetailSerializer
(без correct_answer!)
← { questions: [...] }
5. POST /api/tests/3/submit/
{ answers: [1, 2, 0, 3, 1] } → enrolled? → так
attempts < 3? → так
for q, a in zip(questions, answers):
 if a == q.correct_answer: correct++
score = correct/total * 100
passed = score >= passing_score
INSERT tests_module_testresult
if passed:
 Certificate.get_or_create
 INSERT certificates_certificate
 (з UUID)
← { score: 85, passed: true }
6. GET /api/certificates/ → filter(student=user)
← список сертифікатів
7. jsPDF.save() (на фронтенді) → PDF на пристрій студента

Зовнішні сервіси

Сервіс	Для чого	Безкоштовний план
Vercel	Хостинг Next.js фронтенду + CDN	Hobby (100 GB трафіку, безлім деплоїв)
Railway / Render	Хостинг Django бекенду (gunicorn)	Render Free (750 год/міс), Railway \$5 кредит
Supabase	PostgreSQL + Auth + Storage + Realtime	500 MB БД, 1 GB storage, 50K активних юзерів/міс
GitHub	Зберігання коду + автодеплой	Безлімітні приватні репо

Змінні середовища (ENV)

Backend (.env)

Змінна	Що це	Приклад
<code>SECRET_KEY</code>	Криптографічний ключ Django	<code>django-insecure-...</code>
<code>DEBUG</code>	Режим розробки (на проді = False)	<code>False</code>
<code>DATABASE_URL</code>	Connection string PostgreSQL	<code>postgresql://user:pass@host:5432/db</code>
<code>ALLOWED_HOSTS</code>	Домени, з яких приймати запити	<code>nexlearn.up.railway.app</code>
<code>CORS_ALLOWED_ORIGINS</code>	Дозволені origin для CORS	<code>https://nexlearn.space</code>
<code>DJANGO_SUPERUSER_USERNAME</code>	Логін адміна для build.sh	<code>admin</code>
<code>DJANGO_SUPERUSER_PASSWORD</code>	Пароль адміна для build.sh	<code>SuperSecret123!</code>
<code>DJANGO_SUPERUSER_EMAIL</code>	Email адміна	<code>admin@nexlearn.space</code>

Frontend (.env.local)

Змінна	Що це	Приклад
<code>NEXT_PUBLIC_API_URL</code>	URL бекенду	<code>https://nexlearn.up.railway.app</code>
<code>NEXT_PUBLIC_SITE_URL</code>	URL фронтенду	<code>https://nexlearn.space</code>

Деплой

Backend на Railway

- Підключити GitHub репозиторій
- Root Directory: `backend/`
- Railway знаходить Procfile : `web: gunicorn config.wsgi --bind 0.0.0.0:$PORT`
- Запускає `build.sh` :
 - `pip install -r requirements.txt`
 - `python manage.py migrate`

- `python manage.py collectstatic`
 - Створення / оновлення superuser-a
5. Додати ENV-змінні через Dashboard
 6. Railway видає URL: `nexlearn.up.railway.app`

Frontend на Vercel

1. Підключити GitHub репозиторій
2. Root Directory: `frontend/`
3. Framework: автоматично визначається як Next.js
4. `vercel.json`: `buildCommand: npm run build`
5. Додати `NEXT_PUBLIC_API_URL`
6. Vercel видає URL: `nexlearn.vercel.app` або власний домен

Database на Supabase

1. Створити проект на `supabase.com`
2. Settings → Database → отримати `Connection string` (URI)
3. Вставити в `DATABASE_URL` Railway
4. Перший `migrate` створює всі таблиці автоматично

Нюанси і фішки

Annotated querysets для оптимізації

Замість N+1 запитів — підрахунок одним запитом через `annotate` :

```
Course.objects.annotate(
    _avg_rating=Avg('ratings__score'),
    _ratings_count=Count('ratings', distinct=True),
    _students_count=Count('enrollments', distinct=True),
)
```

Один SQL замість кількох сотень.

Окремий serializer для студентів

`TestStudentDetailSerializer` не містить `correct_answer` . Студент фізично не може отримати правильні відповіді з API.

Ротація refresh-токенів

При кожному `/api/auth/refresh/` — старий refresh додається в blacklist, видається новий. Якщо хтось вкраде refresh — у нього є тільки 7 днів і одна спроба.

Автостворення superuser у CI/CD

`build.sh` створює адміна при першому деплої. При наступних — оновлює пароль (зручно якщо забув).

get_or_create для сертифікатів

Запобігає дублюванню: якщо студент здав тест 5 разів — сертифікат буде один (через `unique_together`).

django-ratelimit на ендпоінтах автентифікації

Замість самописного rate-limit — готова бібліотека. `5/m per IP` на login, `3/m` на register.

WhiteNoise для статички

Не потрібен Nginx — gunicorn сам віддає Django admin CSS/JS через middleware.

JWT-токен у заголовку, не в cookies

Простіше для SPA + не потрібен CSRF. Мінус: вразливо до XSS.

Лінк деактивації admin URL

`/mgmt-panel-x9k2/` замість `/admin/` — мінімальна спроба сховати від ботів-сканерів.

Зв'язки між компонентами

Стрілки: хто куди ходить

Frontend (Next.js → Backend API)

Next.js	—POST→	/api/auth/register/	Реєстрація
Next.js	—POST→	/api/auth/login/	Логін → JWT
Next.js	—POST→	/api/auth/refresh/	Оновити access
Next.js	—GET→	/api/auth/me/	Дані юзера
Next.js	—PATCH→	/api/auth/me/	Редагувати профіль
Next.js	—GET→	/api/courses/	Каталог курсів
Next.js	—POST→	/api/courses/	Створити курс
Next.js	—GET→	/api/courses/{id}/	Деталі курсу
Next.js	—POST→	/api/courses/{id}/enroll/	Записатися
Next.js	—GET→	/api/courses/{id}/progress/	Прогрес
Next.js	—POST→	/api/courses/{id}/materials/	Матеріал (FormData)
Next.js	—POST→	/api/courses/{id}/comments/	Коментар
Next.js	—POST→	/api/courses/{id}/rate/	Оцінка
Next.js	—GET→	/api/tests/	Список тестів
Next.js	—GET→	/api/tests/{id}/	Тест (без correct_answer для студента)
Next.js	—POST→	/api/tests/{id}/submit/	Здати тест
Next.js	—GET→	/api/tests/{id}/export/	CSV (тільки teacher/admin)
Next.js	—GET→	/api/certificates/	Мої сертифікати
Next.js	—GET→	/api/certificates/{id}/	Дані для PDF

Backend (Django → External services)

Django	—SQL→	Supabase PostgreSQL	Усі CRUD-операції
Django	—FS→	media/avatars/	Аватари юзерів
Django	—FS→	media/materials/	Матеріали курсів
Django	—HTTP→	Vercel CDN	(відсутня – фронт окремо)
Django	—SMTP→	(не використовується)	email-розсилки

Frontend (Next.js → Browser APIs)

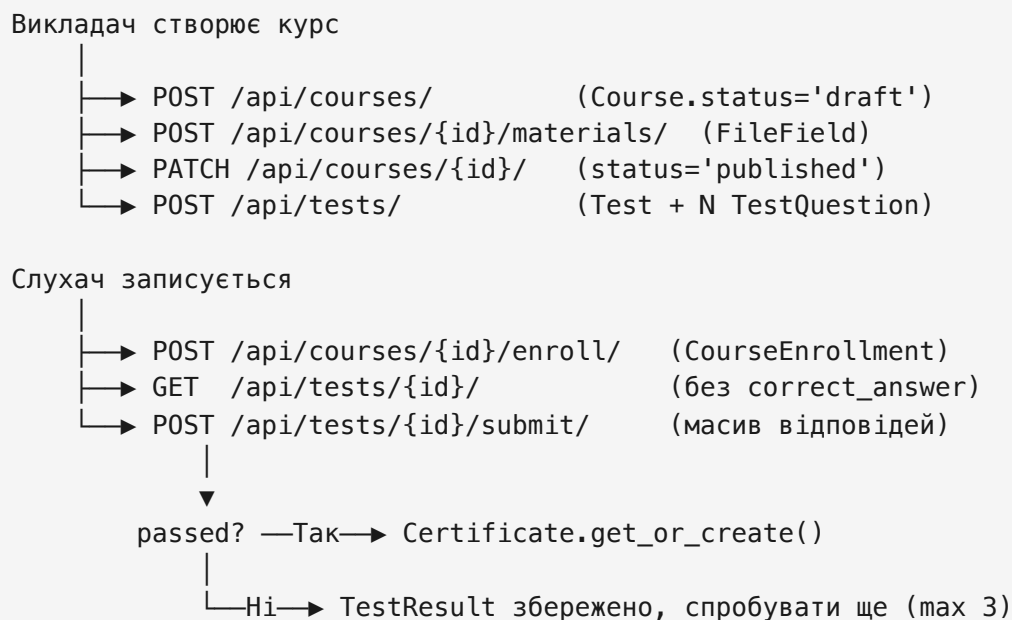
Next.js	—Storage→	localStorage	access_token, refresh_token
Next.js	—jsPDF→	Browser → Disk	PDF сертифіката
Next.js	—Fetch→	Vercel CDN	Статика (Next.js bundle)

Цикли даних

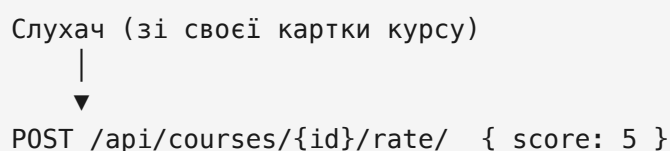
Цикл 1: Авторизація

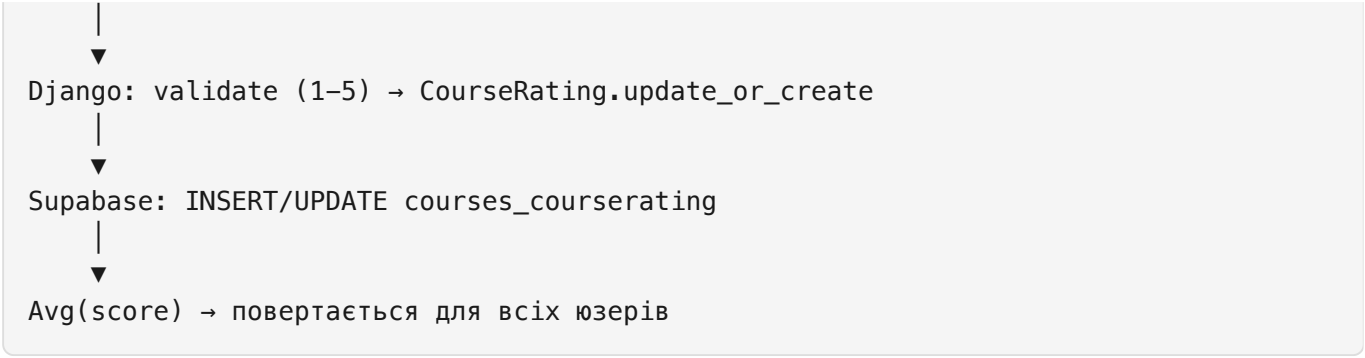


Цикл 2: Курс → Тест → Сертифікат



Цикл 3: Оцінка курсу





Залежності: що від чого залежить

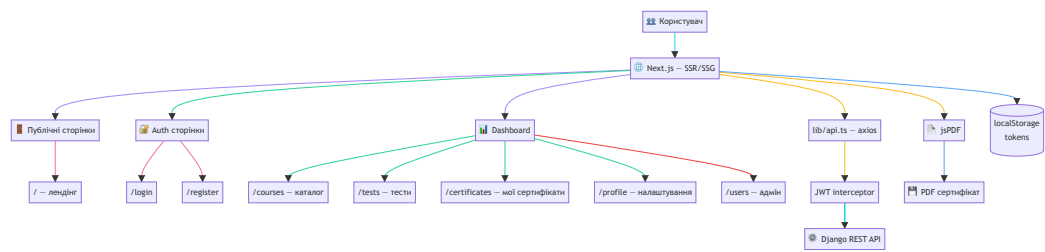
Якщо впаде...	Що зламається
Supabase (PostgreSQL)	Все: ні логіну, ні читання курсів, ні тестів. Сайт показує помилки.
Railway (Backend)	Фронтенд відкривається, але всі API повертають 502/таймаут. Можна тільки лендінг переглянути.
Vercel (Frontend)	Сайт недоступний. Бекенд працює — можна звертатись напряду через REST-клієнт.
GitHub	Не можна задеплоїти оновлення. Поточна версія працює.
Disk (media/)	Аватари і матеріали недоступні. Решта функцій працює.

Токени і ключі

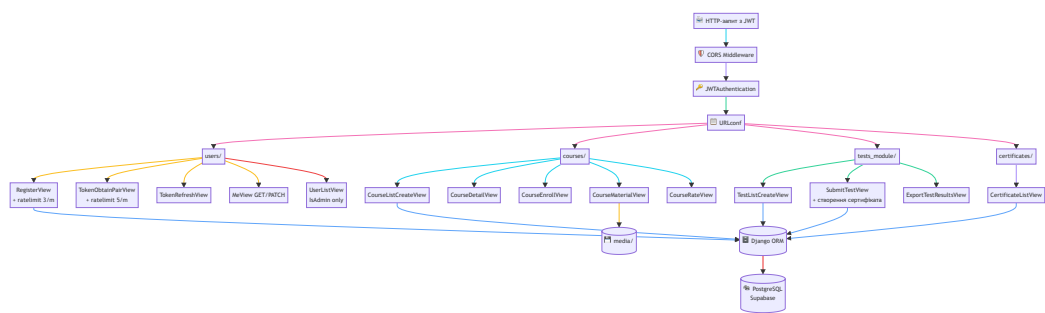
Ключ	Backend	Frontend
SECRET_KEY	✓ (підпис JWT)	×
DATABASE_URL	✓ (підключення)	×
ALLOWED_HOSTS	✓ (захист)	×
CORS_ALLOWED_ORIGINS	✓ (CORS)	×
NEXT_PUBLIC_API_URL	×	✓ (URL бекенду)
access_token (JWT)	verify (стейтлес)	localStorage
refresh_token (JWT)	blacklist у БД	localStorage
DJANGO_SUPERUSER_*	✓ (build.sh)	×

Діаграми зв'язків

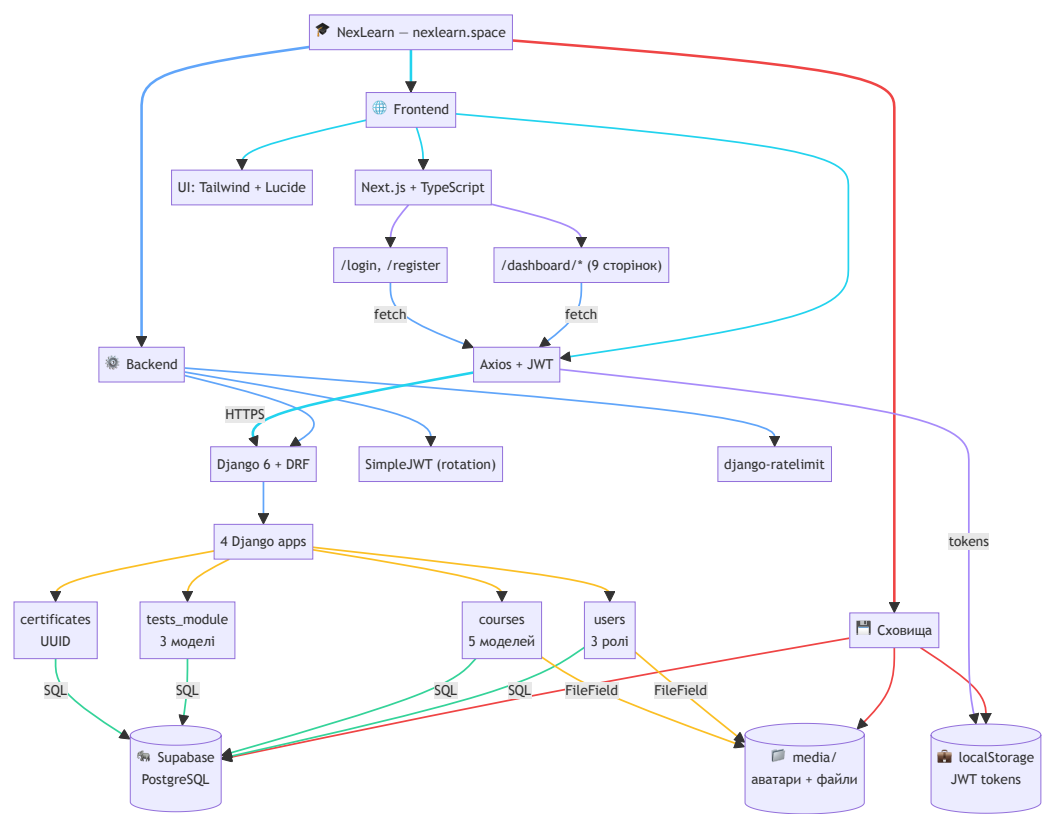
1. Frontend — дерево



2. Backend — дерево



3. Все разом — повне дерево



Порівняння: Frontend vs Backend

Функція	🌐 Frontend (Next.js)	⚙️ Backend (Django)
Реєстрація	Форма з валідацією на льоту	POST /api/auth/register/ + rate-limit + хешування
Вхід	Форма + збереження JWT	SimpleJWT генерує access + refresh
Поновлення сесії	Axios interceptor: автоматичний refresh при 401	Ротація: новий refresh + blacklist старого
Каталог курсів	Список з пошуком, фільтрами, рейтингом	QuerySet з annotate (avg, count)
Створення курсу	Форма з draft/published статусом	IsTeacherOrAdmin permission
Завантаження файлу	FormData + drag-and-drop	FileField з валідацією size ≤ 10MB
Запис на курс	Кнопка з оптимістичним апдейтом	get_or_create на CourseEnrollment
Тестування	UI з таймером, прогрес-баром, навігацією	Підрахунок балів + автостворення сертифіката
Захист від шахрайства	(не покладатись на фронт)	Окремі serializers без correct_answer
Сертифікати	Список + jsPDF для генерації	UUID-номер + автостворення при passed
Адмін-панель	/dashboard/users — змінити роль	Django Admin на /mgmt-panel-x9k2/ + /api/users/
Зберігання токенів	localStorage	Stateless (з blacklist для refresh)
Профіль і аватар	Форма з прев'ю картинки	ImageField + Pillow + 2MB ліміт
Локалізація	Український інтерфейс	LANGUAGE_CODE='uk' , TIME_ZONE='Europe/Kyiv'
Обробка помилок	error.tsx (Next.js error boundary)	DRF exceptions → JSON з детальним описом
Хостинг	Vercel (CDN, безкоштовно)	Railway/Render (gunicorn)
Мова	TypeScript + React 19	Python 3.12 + Django 6

Що спільне

- **Один JWT-токен** — обидва формати працюють з ним (фронт зберігає, бек перевіряє)
- **JSON** — формат обміну даних
- **Ролі** — і фронт, і бек перевіряють роль (фронт ховає елементи UI, бек блокує API)
- **Українська локаль** — інтерфейс і повідомлення
- **Деплой через GitHub** — обидва автодеплоюються при push

Що унікальне

Тільки Frontend	Тільки Backend
SSR/SSG для SEO	Хешування паролів (PBKDF2)
jsPDF — генерація PDF у браузері	Підрахунок балів тесту
localStorage — токени	Видача сертифікатів (UUID)
Tailwind UI	Django Admin (для випадків)
Axios interceptors (auto-refresh)	Rate-limit на ендпоінтах
Lucide іконки	JWT blacklist при ротації
Drag-and-drop файлів	QuerySet annotations

Глосарій термінів

Пояснення всіх технічних слів, які зустрічаються в документі, простою мовою.

Термін	Що це означає
Next.js	Фреймворк на основі React. Дає готову маршрутизацію, сервер-сайд рендеринг (SSR), оптимізацію картинок, API-роути «з коробки». Зараз використовуємо App Router (нова система).
React	JavaScript-бібліотека для будівництва інтерфейсів з компонентів. Кожна кнопка, форма, картка — це React-компонент.
TypeScript	JavaScript з типами. Перед запуском перевіряє: чи правильно ти передаєш дані. Зменшує кількість багів у 5-10 разів.
Tailwind CSS	Бібліотека стилів через класи. Замість писати свій CSS — використовуєш готові класи: <code>className="text-xl font-bold p-4 bg-blue-500"</code> .
Django	Веб-фреймворк на Python. «Батарейки в комплекті» — admin-панель, ORM, автентифікація, форми вже готові.
Django REST Framework (DRF)	Розширення Django для будівництва REST API. Дає Serializers, ViewSets, Permissions, автогенерацію документації.
REST API	Стиль API: ресурси через URL (<code>/api/courses/</code>), методи через HTTP (GET/POST/PUT/DELETE). Дані у JSON.
JWT (JSON Web Token)	Підписаний токен, який містить дані користувача. Бекенд може перевірити його без звернення до БД (stateless).
Access token	Короткий JWT (60 хв) для доступу до API. Передається в заголовок <code>Authorization: Bearer <token></code> .
Refresh token	Довгий JWT (7 днів) для отримання нового access. При використанні старий додається в blacklist.
Token rotation	При кожному оновленні access — видається ще і новий refresh. Старий більше не працює. Захист від крадіжки.
Token blacklist	Список refresh-токенів, які вже не дійсні (через ротацію або logout). Зберігається в БД.
ORM (Object-Relational Mapping)	Замість писати SQL — пишеш Python-код: <code>Course.objects.filter(status='published')</code> . Django сам формує SQL.
Migration	Файл, який описує зміну в БД (додав поле, створив таблицю). Генерується автоматично через <code>makemigrations</code> .
Serializer (DRF)	«Перекладач» між Python-об'єктами і JSON. Описує які поля видавати, які приймати, як валідувати.
Permission	Перевірка «чи має право цей юзер виконати цю дію». Окрема для кожного view.
QuerySet	«Запит до БД» в Django ORM. Лінійний — виконується тільки коли потрібен результат.
annotate()	Метод QuerySet — додає обчислюване поле через SQL-функції (Avg, Count, Sum). Замість N запитів — один.

select_related / prefetch_related	Оптимізація: завантажити пов'язані об'єкти одним запитом замість багатьох (вирішує проблему N+1).
Vercel	Хостинг для Next.js + статичних сайтів. Безкоштовний для невеликих проєктів. Автодеплой з GitHub.
Railway / Render	PaaS-хостинг для Django/Node/Python. Запускає Procfile , надає URL і HTTPS.
Supabase	«Firebase з відкритим кодом». Дає PostgreSQL + Auth + Storage + Realtime. Безкоштовний план для невеликих проєктів.
PostgreSQL	Реляційна база даних. Швидка, надійна, з підтримкою JSON-полів, full-text search, складних запитів.
gunicorn	WSGI-сервер для Python. Запускає Django у виробничому режимі (на проді замість runserver).
WSGI	Стандарт інтерфейсу між веб-сервером і Python-додатком. Django, Flask його використовують.
WhiteNoise	Бібліотека, яка дозволяє Django (через gunicorn) віддавати статичні файли без Nginx.
Procfile	Файл для PaaS (Railway, Render, Heroku) — описує які процеси запускати: web: gunicorn
CORS (Cross-Origin Resource Sharing)	Правило безпеки браузера: API дозволяє запити тільки з вказаних доменів. Захист від чужих сайтів.
CSRF (Cross-Site Request Forgery)	Атака, коли чужий сайт «підпихає» запити до твого API. Захист — токен в формі. Не потрібен якщо JWT в заголовку.
CSP (Content Security Policy)	Заголовок, який вказує браузеру що завантажувати дозволено (свій код, свої картинки). Блокує XSS-атаки.
HSTS (HTTP Strict Transport Security)	Заголовок, який каже браузеру: завжди використовуй HTTPS для цього домену протягом N днів.
XSS (Cross-Site Scripting)	Атака: чужий JavaScript-код виконується на твоєму сайті. Захист: CSP + санітизація вводу.
Rate-limit	Обмеження запитів. Наприклад, «5 спроб логіну на хвилину з однієї IP». Захист від брутфорсу.
PBKDF2	Алгоритм хешування паролів. Дуже повільний — навіть якщо вкрати БД, підбір одного пароля займає місяці.
UUID	128-бітний унікальний ідентифікатор: 550e8400-e29b-41d4-a716-446655440000 . Шанс колізії практично нуль.
Axios	HTTP-бібліотека для JavaScript. Простіший за fetch , підтримує interceptors (для auto-refresh JWT).
jsPDF	JavaScript-бібліотека для генерації PDF у браузері. Не потрібен сервер.
localStorage	Сховище в браузері (~5 MB). Тут зберігаються JWT-токени. Доступ через localStorage.setItem(...) .
SSR (Server-Side Rendering)	Next.js рендерить HTML на сервері і віддає браузеру. Швидше перше завантаження, краще для SEO.

SSG (Static Site Generation)	Сторінки генеруються на етапі білду. Vercel CDN віддає їх миттєво. Найшвидший варіант.
SPA (Single Page Application)	Сайт, який не перезавантажує сторінку. Усе через React + AJAX. Швидко, плавно.
App Router (Next.js)	Нова система маршрутизації Next.js (з v13). Папки в <code>src/app/</code> = маршрути.
Middleware	Код, який виконується між запитом і відповіддю. Django: CORS, AuthN, GZip. Next.js: edge-функції.
django-ratelimit	Бібліотека для обмеження запитів у Django. Додається через декоратор: <code>@ratelimit(key='ip', rate='5/m')</code> .
django-cors-headers	Бібліотека для CORS у Django. Додає правильні заголовки для дозволених origin-ів.
Pillow	Бібліотека для обробки картинок у Python. Потрібна Django для ImageField (валідація формату, ресайз).
FileField / ImageField	Поля моделі Django для зберігання файлів. Файл лежить на диску (або S3), у БД — тільки шлях.

Маршрут запиту від А до Я

Покрокове пояснення що відбувається при кожній дії — від натискання кнопки до відповіді.

Сценарій 1: Слухач реєструється і входить

Крок	Де	Що відбувається
1	Браузер	Слухач відкриває <code>nexlearn.space/register</code>
2	Vercel CDN	Віддає згенеровану Next.js сторінку
3	React	Гідрація: HTML стає інтерактивним. Малюється форма
4	Браузер	Слухач заповнює: username, email, password, role=student
5	Axios	POST <code>{API_URL}/api/auth/register/</code> з JSON
6	CORS Middleware	Перевіряє <code>Origin</code> заголовок проти <code>CORS_ALLOWED_ORIGINS</code>
7	django-ratelimit	Перевіряє: чи не більше 3 спроб реєстрації з цієї IP за хвилину
8	RegisterSerializer	Валідація: паролі співпадають, role не admin, username унікальний
9	Django ORM	<code>set_password</code> хешує через PBKDF2 → INSERT users_user
10	Supabase	Запис у таблиці <code>users_user</code>
11	Frontend	Отримує { id, username, role, ... } → редірект на /login
12	Браузер	Слухач вводить пароль → POST <code>/api/auth/login/</code>
13	SimpleJWT	authenticate() → генерує access (60 хв) + refresh (7 дн)
14	Frontend	<code>localStorage.setItem('access_token', ...)</code>
15	Браузер	Редірект на <code>/dashboard</code>

Сценарій 2: Викладач створює курс і додає матеріал

Крок	Де	Що відбувається
1	Браузер	Викладач на <code>/dashboard/courses</code> натискає «Створити курс»
2	Frontend	POST <code>/api/courses/</code> з { title, description, status: 'draft' }
3	Axios	Додає <code>Authorization: Bearer <access_token></code>
4	JWT Auth	Перевіряє підпис, decode payload → request.user
5	Permissions	<code>IsTeacherOrAdmin</code> → user.role == 'teacher' ✓

6	Serializer	<code>CourseCreateSerializer</code> валідує + автозаповнює <code>teacher = request.user</code>
7	Django ORM	INSERT courses_course → повертає id=5
8	Frontend	Редірект на /dashboard/courses/5
9	Браузер	Drag-and-drop файла lecture.pdf (8 MB)
10	Axios	POST /api/courses/5/materials/ з FormData (multipart)
11	Permissions	<code>IsOwnerOrAdmin</code> → course.teacher == request.user ✓
12	Validator	<code>validate_file_size_10mb</code> → 8 MB < 10 MB ✓
13	Django	FileField зберігає в <code>media/materials/lecture_abc123.pdf</code>
14	Supabase	INSERT courses_coursematerial з шляхом до файлу
15	Frontend	Показує файл у списку матеріалів

Сценарій 3: Слухач здає тест і отримує сертифікат

Крок	Де	Що відбувається
1	Браузер	Слухач відкриває <code>/dashboard/tests/3</code>
2	Frontend	GET /api/tests/3/ з JWT
3	Backend	user.role == 'student' → <code>TestStudentDetailSerializer</code> (без correct_answer!)
4	Frontend	Малює запитання + варіанти + таймер на time_limit хвилин
5	Браузер	Слухач відповідає, накопичується <code>answers = [1, 2, 0, 3, 1]</code>
6	Frontend	POST /api/tests/3/submit/ з масивом
7	SubmitTestView	Перевіряє <code>CourseEnrollment.exists()</code> → ✓
8	SubmitTestView	Перевіряє <code>TestResult.count() < 3</code> → ✓ (це перша спроба)
9	SubmitTestView	Перевіряє довжина масиву == questions.count()
10	SubmitTestView	Цикл: <code>for q, a in zip(...): if a == q.correct_answer: correct++</code>
11	SubmitTestView	<code>score = round(correct/total * 100) = 80</code>
12	SubmitTestView	<code>passed = 80 >= 70</code> (passing_score) → ✓
13	Django ORM	INSERT tests_module_testresult
14	Django ORM	<code>Certificate.get_or_create</code> → INSERT certificates_certificate з UUID
15	Frontend	Отримує { score: 80, passed: true } → показує модальне вікно

16	Браузер	Слухач переходить на <code>/dashboard/certificates</code>
17	Frontend	GET <code>/api/certificates/</code> → список
18	jsPDF	Натискає «Завантажити PDF» → генерує A4 з даними сертифіката
19	Браузер	Зберігає <code>certificate-<code>{uuid}</code>.pdf</code> на пристрій

Сценарій 4: Адмін змінює роль користувача

Крок	Де	Що відбувається
1	Браузер	Адмін відкриває <code>/dashboard/users</code>
2	Frontend	GET <code>/api/users/</code> з JWT
3	Permissions	<code>IsAdmin</code> → <code>user.is_admin</code> ✓ (інакше 403)
4	Backend	QuerySet з пагінацією → <code>UserListSerializer</code>
5	Frontend	Малює таблицю з юзерами і селектами ролей
6	Браузер	Адмін обирає для юзера <code>id=42</code> нову роль <code>'teacher'</code>
7	Frontend	PATCH <code>/api/users/42/role/</code> з <code>{ role: 'teacher' }</code>
8	UserChangeRoleView	<code>ChangeRoleSerializer</code> валідує: <code>role ∈ {admin, teacher, student}</code>
9	Django ORM	<code>user.role = 'teacher'; user.save(update_fields=['role'])</code>
10	Supabase	UPDATE <code>users_user</code> SET <code>role='teacher'</code> WHERE <code>id=42</code>
11	Frontend	Оновлений <code>UserSerializer</code> → перерендер UI

Сценарії помилок і як система їх обробляє

Що сталося	Що відбувається	Як система реагує
JWT прострочений	Access-токен старший за 60 хв, юзер робить запит	Backend повертає 401. Axios interceptor ловить, робить POST /api/auth/refresh/ з refresh-токеном, отримує новий access, повторює оригінальний запит. Користувач нічого не помічає.
Refresh теж прострочений	Refresh старший за 7 днів	Refresh повертає 401. Frontend очищує localStorage і редіректить на /login. Користувач знову логіниться.
Брутфорс пароля	Хтось пробує підібрати пароль перебором	django-ratelimit: 5 спроб логіну на хвилину з однієї IP. Після 5-ї — 429 Too Many Requests на хвилину. Заборона спроб через декоратор.
Файл понад 10 MB	Викладач пробує завантажити велике відео як матеріал	validate_file_size_10mb повертає ValidationError. Серіалізатор повертає 400 з повідомленням «Максимальний розмір файлу — 10MB». Файл не зберігається.
Слухач не записаний на курс	Пробує здати тест курсу, на який не записався	SubmitTestView перевіряє CourseEnrollment.exists() → 403 Forbidden з повідомленням «Ви не записані на цей курс».
Перевищено 3 спроби	Слухач здає тест 4-й раз	TestResult.count() >= 3 → 400 Bad Request з повідомленням «Ви вичерпали кількість спроб». Записати результат не дозволяє.
Студент пробує редагувати чужий курс	PATCH /api/courses/5/ від юзера, який не власник	IsOwnerOrAdmin перевіряє obj.teacher == request.user → 403 Forbidden. Запис не оновлюється.
Реєстрація як admin	Зловмисник передає {"role": "admin"} в /api/auth/register/	RegisterSerializer.validate_role ловить → ValidationError «Неможливо зареєструватися як адміністратор». Тільки існуючий адмін може видати цю роль.
CORS-помилка	Запит з чужого сайту (наприклад, evil.com) до nexlearn API	Backend не додає Access-Control-Allow-Origin, бо origin не в CORS_ALLOWED_ORIGINS. Браузер блокує відповідь. Запит фактично пройшов, але JS не може прочитати результат.
Supabase недоступний	Database connection timeout	Django ORM кидає OperationalError. DRF загортає у 500 Internal Server Error. Frontend показує «Сервер тимчасово недоступний». Конкретний запит провалюється, інші теж — нічого не працює.

Дублювання запису на курс	Слухач натискає «Записатися» двічі швидко	<code>get_or_create</code> створює тільки один запис. Другий виклик повертає 400 «Ви вже записані на цей курс».
Невалідний JSON	Frontend надсилає поламаний JSON (баг)	DRF parser повертає 400 Bad Request з описом проблеми парсингу.
Втрата refresh-токена через ротацію	Юзер відкрив сайт у двох вкладках, обидві оновлюють токен одночасно	Перша вкладка отримує новий refresh, друга з потім старий — старий вже в blacklist → 401. Друга вкладка робить логат. (Для бойового — можна зберігати tokens в одному джерелі через BroadcastChannel API.)
SQL Injection	Зловмисник передає <code>' ; DROP TABLE users; --</code> у поле	Django ORM використовує параметризовані запити — текст не виконується як SQL. Просто зберігається як рядок (або відхиляється валідацією).
XSS-атака	Юзер у коментарі пише <code><script>alert('xss')</script></code>	React за замовчуванням екранує текст. Скрипт відображається як текст, не виконується. Плюс CSP блокує inline-скрипти.

Родмап розвитку проекту

Ідеї та плани на майбутнє — що можна додати для покращення NexLearn.

Короткострокові (легко додати)

Фіча	Опис	Де
Темна тема	Toggle у профілі. Зберігати у localStorage. Tailwind має <code>dark:</code> модифікатор	Frontend
Email-підтвердження	При реєстрації — лист з посиланням. Активація акаунта тільки після кліку	Backend + Frontend
Скидання пароля	«Забули пароль?» → email з токеном → форма нового пароля	Backend + Frontend
Прогрес-бар тесту	Показувати «Запитання 3 з 10» і таймер до завершення	Frontend
Сортування і фільтри курсів	За рейтингом, датою, кількістю слухачів. Фільтр за категорією	Backend + Frontend

Середньострокові (потребують роботи)

Фіча	Опис	Де
S3 / Cloudinary для файлів	Замість локального media/ — django-storages з S3. Файли переживуть рестарт контейнера	Backend
Категорії курсів	Окрема модель Category, ManyToMany з Course. Фільтрація і навігація	Backend + Frontend
Розширені типи запитань	Multiple choice (декілька правильних), Open answer (текст), True/False	Backend + Frontend
Часові обмеження тесту (server-side)	Зараз time_limit тільки в UI. Додати на бекенді: фіксувати started_at, відкидати submit після ліміту	Backend
Прогрес курсу візуально	Шкала «3 із 5 тестів пройдено». Бейджи за етапи	Frontend
Чат викладач-слухач	Особисті повідомлення в курсі. Через WebSockets або polling	Backend + Frontend
Аналітика для викладача	Дашборд: середній бал по тестах, найскладніше запитання, графік активності	Backend + Frontend
Експорт сертифіката на бекенді	Замість jsPDF — Django + WeasyPrint → красиві PDF з шрифтами і підписами	Backend

 Довгострокові (великі проекти)

Фіча	Опис	Де
Відео-уроки	Завантаження відео + плеер з контролем перегляду. HLS-стрімінг через Cloudflare	Frontend + Backend
Платні курси	Інтеграція з LiqPay / Stripe. Платіжні плани, підписки. Доступ після оплати	Backend + Frontend
Мобільний додаток	React Native або Flutter. Push-сповіщення про нові курси, тести, результати	Новий проект
AI-помічник	GPT для генерації запитань тесту по матеріалу курсу. Аналіз відкритих відповідей	Backend
Live-вебінари	Інтеграція з Zoom/Jitsi. Розклад вебінарів у курсі. Автоматичні нагадування	Backend + Frontend
Гейміфікація	Бейджи, рівні, лідерборди, рейтинги слухачів за активністю	Backend + Frontend
Багатомовність	Інтерфейс UA/EN/RU. Курси з перекладами. Django i18n + Next.js i18n	Backend + Frontend
Інтеграція з SCORM	Імпорт навчального контенту в стандарті SCORM (важливо для корпоративного навчання)	Backend

Пріоритети

Пріоритет	Фічі	Чому
 Високий	Скидання пароля, email-підтвердження, S3 для файлів	Базові вимоги для production. Без них — критичні баги при масштабуванні
 Середній	Розширені типи запитань, чат, категорії, аналітика	Покращують UX, конкурентоспроможність на ринку онлайн-навчання
 Низький	Відео-уроки, AI, мобільний додаток, SCORM	Великі проекти з окремим бюджетом і командою

Шари безпеки — від зовнішнього до внутрішнього

Система захищена на кожному рівні, як цибуля — кожен шар додає свій захист.



Інтернет — зовнішній світ



Шар 1: HTTPS + HSTS



Шар 2: CORS + CSP



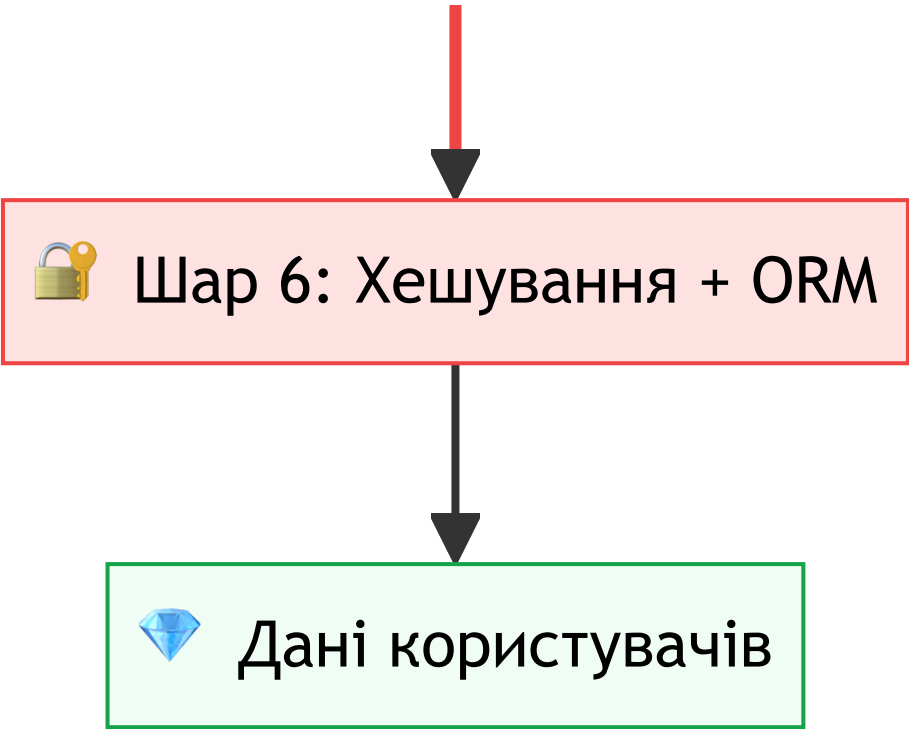
Шар 3: Rate-limit



Шар 4: JWT-автентифікація



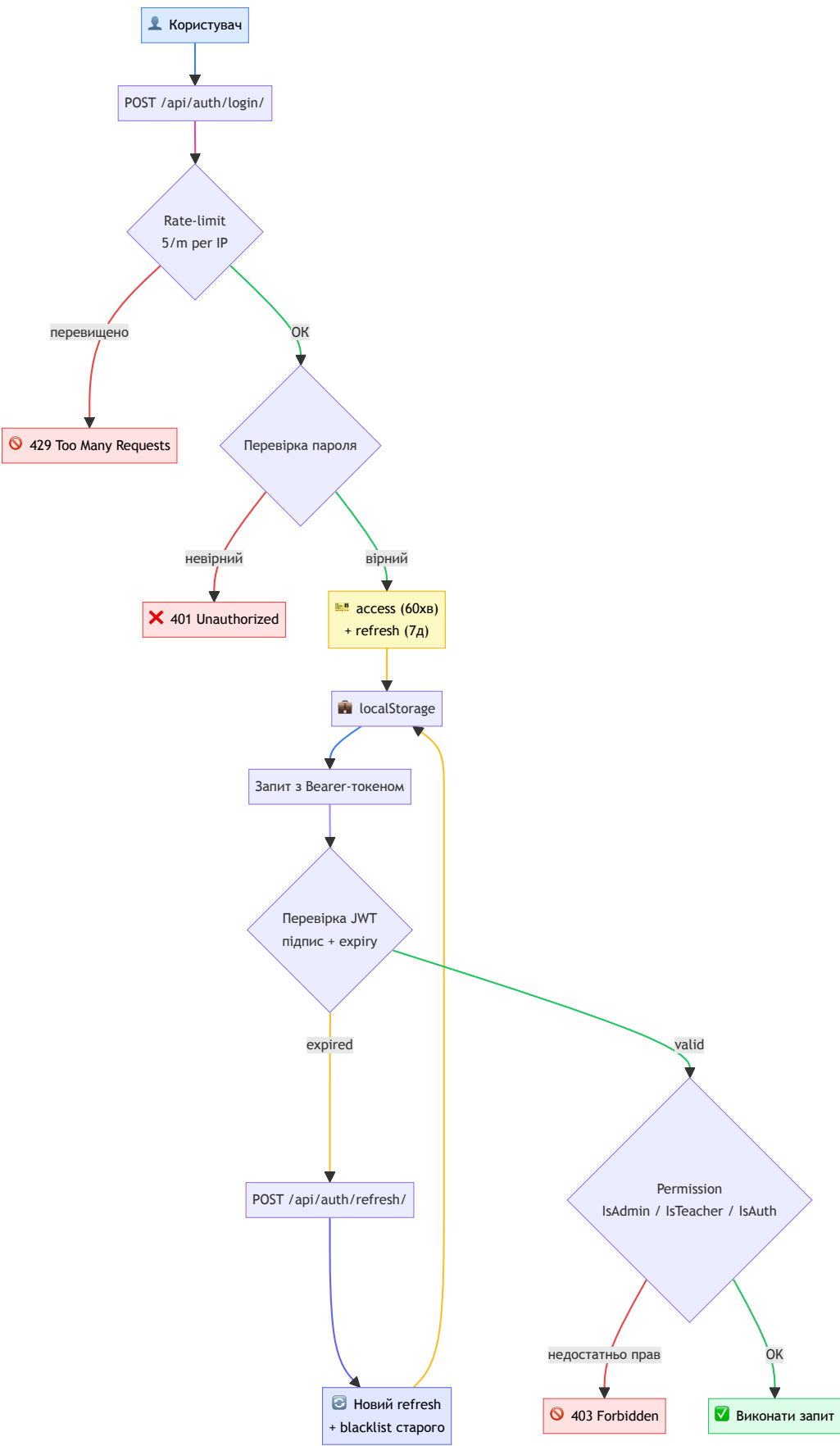
Шар 5: Авторизація ролей



Деталі кожного шару

#	Шар	Що захищає	Як працює
1	HTTPS + HSTS	Весь трафік між браузером і серверами	TLS-шифрування — ніхто не може прочитати запити. HSTS : браузер примусово використовує HTTPS на 1 рік. Vercel і Railway надають автоматично.
2	CORS + CSP	Захист від чужих сайтів і XSS	CORS : backend дозволяє запити тільки з <code>nexlearn.space</code> . CSP : frontend забороняє чужі скрипти, дозволяє тільки <code>self</code> і API URL.
3	Rate-limit	Захист від спаму і брутфорса	django-ratelimit: <code>login = 5/m per IP</code> , <code>register = 3/m per IP</code> . Блокування в кеші Django (LocMemCache).
4	JWT-автентифікація	Підтвердження особи на кожному запиті	Access-токен (60 хв) + refresh (7 днів). Підпис HMAC-SHA256 з SECRET_KEY. Ротація + blacklist при оновленні. Без токена → 401.
5	Авторизація ролей	Хто що може робити	Слухач : дивитися курси, складати тести. Викладач : + створювати свої курси. Адмін : + усе. Перевірка через DRF Permissions на кожному view.
6	Хешування + ORM	Захист збережених даних	Паролі : PBKDF2-SHA256 (Django default). SQL Injection : захист через Django ORM (параметризовані запити). UUID : для номерів сертифікатів — неможливо вгадати.

Захист JWT-флоу

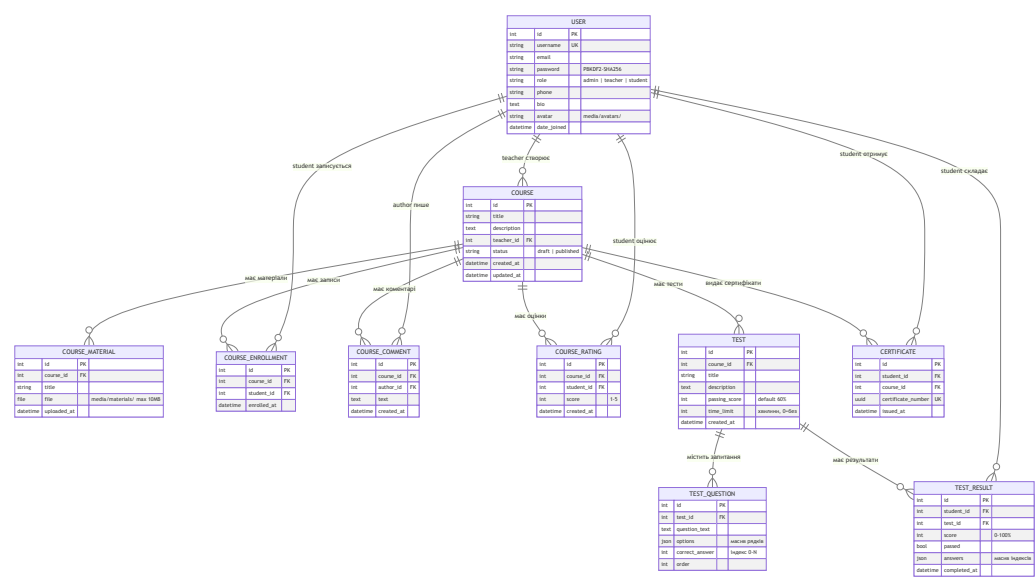


Що захищено і чим — підсумок

Що	Загроза	Захист
Паролі в БД	Витік дампа	PBKDF2-SHA256 (Django default) — підбір одного пароля = місяці
JWT-токени	Перехоплення	HTTPS + короткий TTL access (60 хв) + ротація refresh + blacklist
Реєстрація як admin	Привілейована атака	<code>RegisterSerializer.validate_role</code> блокує admin-роль
API-ендпоінти	Спам / брутфорс	django-ratelimit per IP + per endpoint
Чужі курси	Несанкціоноване редагування	IsOwnerOrAdmin перевіряє <code>obj.teacher == request.user</code>
Шахрайство в тесті	Підглядання правильних відповідей	Окремий serializer без correct_answer для слухачів
Дублювання сертифікатів	Юзер пробує отримати декілька сертифікатів	<code>unique_together = ['student', 'course'] + get_or_create</code>
SQL Injection	Виконання чужого SQL	Django ORM (параметризовані запити)
XSS-атаки	Вставка чужого JS у коментарі	React екранує текст + CSP блокує inline-скрипти
CSRF	Запити з чужого сайту	JWT-only API (немає cookie-сесії), CORS allowlist
Clickjacking	Вбудовування сайту в iframe	X-Frame-Options: DENY
Великі файли	DDoS через uploads	Validators (2MB аватар, 10MB матеріал)
Чужий доступ до /api/users/	Перегляд усіх юзерів	IsAdmin permission + secret URL для admin-панелі

ER-діаграма бази даних

Повна схема моделей з полями, типами і зв'язками між таблицями PostgreSQL (Supabase).



Обмеження цілісності (constraints)

Таблиця	Constraint	Що забезпечує
CourseEnrollment	unique_together: [course, student]	Студент записується на курс тільки один раз
CourseRating	unique_together: [course, student]	Одна оцінка на курс від студента
Certificate	unique_together: [student, course]	Один сертифікат на курс для студента
Certificate	certificate_number: unique UUID	Унікальний номер кожного сертифіката
TestResult	idx_result_student_test	Індекс для швидкого пошуку спроб студента
TestResult	idx_result_student_passed	Індекс для фільтрації успішних спроб
Course.status	db_index=True	Швидка фільтрація published/draft

Каскадне видалення

Усі зовнішні ключі мають `on_delete=CASCADE` :

- Видалення **User** → видаляються його курси, записи, коментарі, оцінки, результати, сертифікати
- Видалення **Course** → видаляються матеріали, записи, коментарі, оцінки, тести, сертифікати курсу
- Видалення **Test** → видаляються запитання і результати тесту

Middleware Pipeline — обробка запиту

Кожен HTTP-запит проходить через ланцюжок middleware у точному порядку. Відповідь проходить у зворотному порядку.



HTTP-запит від браузера



1. SecurityMiddleware
HTTPS redirect, HSTS



2. WhiteNoiseMiddleware
Статика Django Admin



3. GZipMiddleware
Стиснення відповідей



4. CorsMiddleware

CORS-заголовки

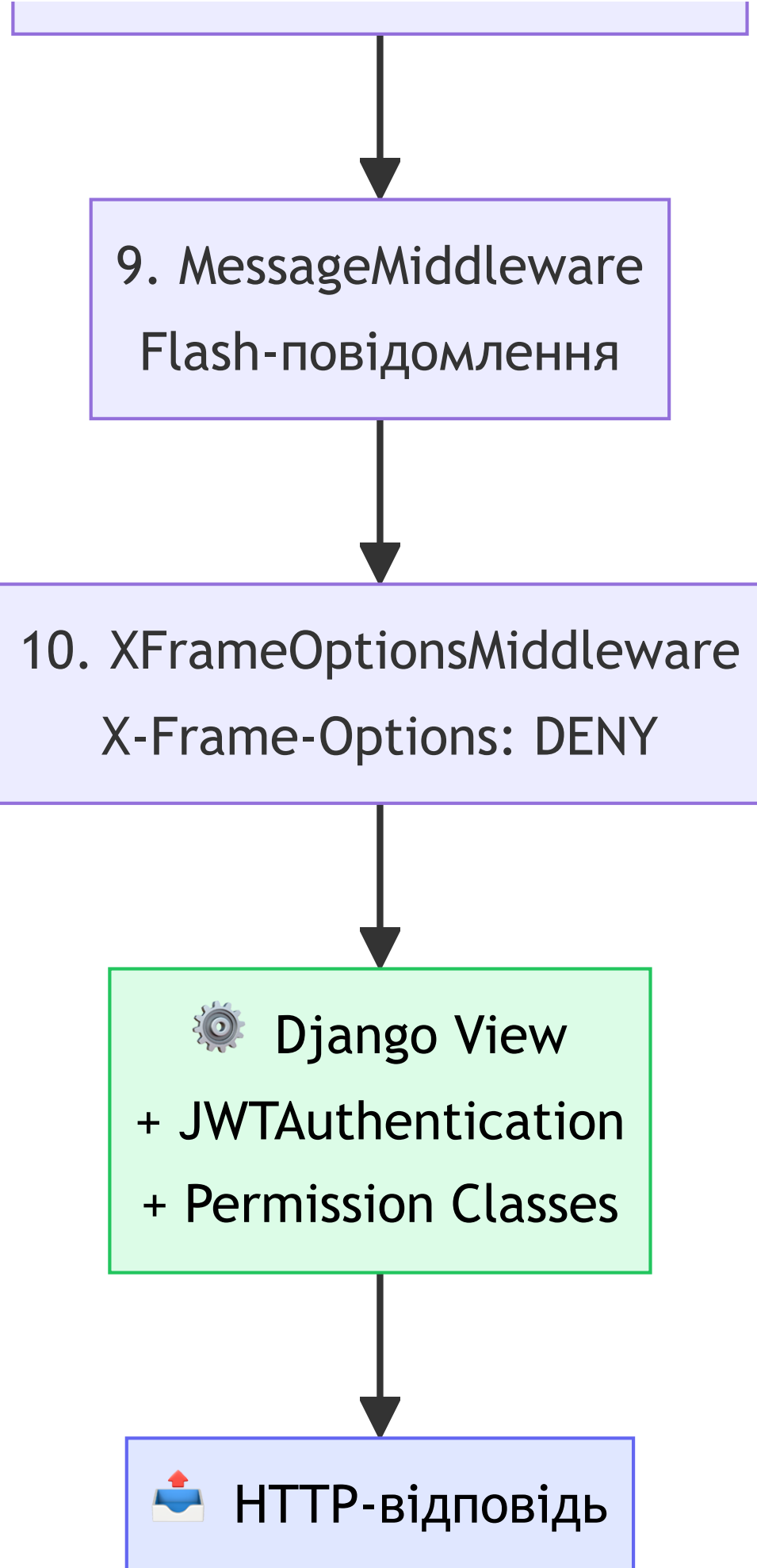
```
graph TD; A[CORS-заголовки] --> B["5. SessionMiddleware  
Сесії для Django Admin"]; B --> C["6. CommonMiddleware  
URL-нормалізація"]; C --> D["7. CsrfViewMiddleware  
CSRF для Django Admin"]; D --> E["8. AuthenticationMiddleware  
Django session auth"];
```

5. SessionMiddleware
Сесії для Django Admin

6. CommonMiddleware
URL-нормалізація

7. CsrfViewMiddleware
CSRF для Django Admin

8. AuthenticationMiddleware
Django session auth



```
graph TD; A[ ] --> B[9. MessageMiddleware<br/>Flash-повідомлення]; B --> C[10. XFrameOptionsMiddleware<br/>X-Frame-Options: DENY]; C --> D[⚙️ Django View<br/>+ JWTAuthentication<br/>+ Permission Classes]; D --> E[📧 HTTP-відповідь];
```

9. MessageMiddleware
Flash-повідомлення

10. XFrameOptionsMiddleware
X-Frame-Options: DENY

⚙️ Django View
+ JWTAuthentication
+ Permission Classes



HTTP-відповідь

Деталі middleware

#	Middleware	Що робить	Для API чи Admin?
1	SecurityMiddleware	HTTPS redirect (<code>SECURE_SSL_REDIRECT</code>), HSTS заголовок (1 рік), <code>X-Content-Type-Options: nosniff</code>	Обидва
2	WhiteNoiseMiddleware	Віддає статичні файли Django Admin (CSS, JS) без Nginx. Стискає + кешує	Тільки Admin
3	GZipMiddleware	Стискає JSON-відповіді (gzip). Економить трафік на 60-80%	Обидва
4	CorsMiddleware	Додає <code>Access-Control-Allow-Origin</code> для дозволених origin. Блокує запити з чужих сайтів	Тільки API
5	SessionMiddleware	Керує сесіями. Для API не використовується (JWT-only), але потрібен для Django Admin	Тільки Admin
6	CommonMiddleware	Нормалізація URL (trailing slash), заборона доступу до прихованих файлів	Обидва
7	CsrfViewMiddleware	CSRF-захист для форм Django Admin. Для API відключено (JWT)	Тільки Admin
8	AuthenticationMiddleware	Заповнює <code>request.user</code> зі сесії. Для API — JWTAuthentication робить це окремо	Тільки Admin
9	MessageMiddleware	Flash-повідомлення типу «Курс успішно створено» для Django Admin	Тільки Admin
10	XFrameOptionsMiddleware	<code>X-Frame-Options: DENY</code> — забороняє вставку в iframe	Обидва

Порядок для API-запиту (типовий GET /api/courses/)

```
Браузер → HTTPS → GZip → CORS ✓ → (Session skip) → (CSRF skip)
  → URL Router → CourseListView
    → JWTAuthentication (Bearer token → request.user)
    → IsAuthenticated permission ✓
    → QuerySet + Serializer → JSON
← GZip стискає → CORS-заголовки → Браузер
```

Конфігурація Django — config/settings.py

Повний розбір ключових налаштувань з поясненнями чому обрано саме такі значення.

REST Framework

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ),
    'DEFAULT_PERMISSION_CLASSES': (
        'rest_framework.permissions.IsAuthenticated', # за замовчуванням — авторизовані
    ),
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 20, # по 20 записів на сторінку
}
```

Параметр	Чому так
JWTAuthentication	Stateless — не потрібен Redis/сесії. Масштабується горизонтально
IsAuthenticated	Безпечний default: відкриті ендпоінти треба явно позначити AllowAny
PAGE_SIZE = 20	Оптимально для мобільних екранів. Не надто багато — не надто мало

SimpleJWT

```
SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=60), # 1 година
    'REFRESH_TOKEN_LIFETIME': timedelta(days=7), # 1 тиждень
    'ROTATE_REFRESH_TOKENS': True, # видавати новий refresh при оновленні
    'BLACKLIST_AFTER_ROTATION': True, # старий refresh — в чорний список
    'AUTH_HEADER_TYPES': ('Bearer',), # Authorization: Bearer
}
```

Параметр	Значення	Чому
Access lifetime	60 хв	Достатньо для тесту (до 30 хв) + запас. Не занадто довго для безпеки
Refresh lifetime	7 днів	Юзер не перелогінується кожен день. Після тижня — знову логін
Rotation	True	Кожне оновлення = новий refresh. Крадій має одну спробу
Blacklist	True	Старий refresh більше не працює — навіть якщо його вкрали

CORS

```
CORS_ALLOWED_ORIGINS = ['https://nexlearn.space'] # тільки наш фронтенд
CORS_ALLOW_CREDENTIALS = True # дозволити cookies (для Django Admin)
CORS_ALLOW_METHODS = ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS']
CORS_ALLOW_HEADERS = ['accept', 'authorization', 'content-type',
                      'origin', 'x-requested-with']
```

Безпека в production

```
# Активується тільки коли DEBUG = False
SECURE_PROXY_SSL_HEADER = ('HTTP_X_FORWARDED_PROTO', 'https') # за проксі Railway
SECURE_SSL_REDIRECT = False # Railway сам робить redirect
SECURE_HSTS_SECONDS = 31536000 # HSTS на 1 рік
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SESSION_COOKIE_SECURE = True # Cookie тільки через HTTPS
CSRF_COOKIE_SECURE = True # CSRF-cookie тільки через HTTPS
```

Файли і ліміти

Параметр	Значення	Що обмежує
DATA_UPLOAD_MAX_MEMORY_SIZE	10 MB	Максимальний розмір тіла POST-запиту
FILE_UPLOAD_MAX_MEMORY_SIZE	10 MB	Максимальний розмір файлу в пам'яті (більше → тимчасовий файл)
MEDIA_ROOT	backend/media/	Директорія для завантажених файлів
STATIC_ROOT	backend/staticfiles/	Директорія для collectstatic (WhiteNoise)
CONN_MAX_AGE	600 сек	Повторне використання з'єднання з БД (10 хв)

Логування

```
LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'formatters': {
        'verbose': {
            'format': '[{asctime}] {levelname} {name} {message}',
            'style': '{',
        },
    },
    'handlers': {
        'console': {
```



```

        'class': 'logging.StreamHandler',
        'formatter': 'verbose',
    },
},
'root': {
    'handlers': ['console'],
    'level': 'INFO',          # Усі логи рівня INFO і вище
},
'loggers': {
    'django.request': {
        'handlers': ['console'],
        'level': 'WARNING',   # Тільки помилки запитів (не кожен 200 OK)
        'propagate': False,
    },
},
}

```

Формат логу: `[2026-05-07 12:34:56] WARNING django.request Not Found: /api/courses/999/`

Railway / Render автоматично збирає stdout і показує у своїй консолі.

Installed Apps — порядок і залежності

Додаток	Тип	Що дає
<code>django.contrib.admin</code>	Django	Вбудована адмін-панель
<code>django.contrib.auth</code>	Django	Користувачі, групи, дозволи
<code>django.contrib.contenttypes</code>	Django	Підтримка generic relations
<code>django.contrib.sessions</code>	Django	Сесії для Django Admin
<code>django.contrib.messages</code>	Django	Flash-повідомлення
<code>django.contrib.staticfiles</code>	Django	Збірка і роздача статичних файлів
<code>rest_framework</code>	Third-party	DRF — REST API
<code>rest_framework_simplejwt</code>	Third-party	JWT-автентифікація
<code>corsheaders</code>	Third-party	CORS-заголовки
<code>rest_framework_simplejwt.token_blacklist</code>	Third-party	Чорний список refresh-токенів
<code>apps.users</code>	Проектний	Кастомний User, реєстрація, логін
<code>apps.courses</code>	Проектний	Курси, матеріали, записи, коментарі, оцінки
<code>apps.tests_module</code>	Проектний	Тести, запитання, результати
<code>apps.certificates</code>	Проектний	Сертифікати з UUID

Підсумок проекту

Що було створено

NexLearn — повноцінна веб-орієнтована інформаційна система для центрів підвищення кваліфікації, яка охоплює весь цикл навчання: від створення курсу викладачем до отримання сертифіката слухачем.

Архітектурні рішення

Рішення	Обґрунтування
Розділення Frontend/Backend	Незалежний деплой, різні команди, масштабування кожної частини окремо
REST API + JWT	Stateless — кожен запит самодостатній, горизонтальне масштабування без сесій
Next.js для фронтенду	SSR для SEO, App Router для зручної маршрутизації, Vercel для безкоштовного хостингу
Django + DRF для бекенду	«Батарейки в комплекті»: ORM, admin, auth, permissions — менше коду, більше безпеки
PostgreSQL на Supabase	Надійна реляційна БД з безкоштовним планом, автобекапи, SSL, connection pooling
JWT-ротація з blacklist	Компроміс між UX (не перелогінюватися) і безпекою (обмежений час життя)
jsPDF для сертифікатів	Генерація PDF у браузері — бекенд не витрачає CPU, не потрібні серверні шрифти
Безкоштовний хостинг	Vercel + Railway/Render + Supabase = 0 € на початковому етапі

Технологічний стек — підсумок

Next.js 16 Frontend фреймворк	Django 6 Backend фреймворк	PostgreSQL База даних	JWT Автентифікація
---	--	---	--

Кількісні показники

Метрика	Значення
Ролі користувачів	3 (admin, teacher, student)
Django-додатки	4 (users, courses, tests_module, certificates)
Моделей даних	10 таблиць у PostgreSQL

API-ендпоінтів	21 (5 auth + 2 users + 12 courses + 7 tests + 2 certificates)
Сторінок фронтенду	9 (лендінг + auth + dashboard pages)
Зовнішні сервіси	3 (Vercel, Railway/Render, Supabase)
Мови програмування	2 (Python 3.12, TypeScript)
Шарів безпеки	6 (HTTPS → CORS → Rate-limit → JWT → Roles → Hashing)
Вартість хостингу	0 € (безкоштовні плани)

Що забезпечує система

- **Створення і публікація курсів** — викладачі наповнюють контент, додають матеріали, публікують
- **Запис на курси** — слухачі обирають курси, записуються, отримують доступ до матеріалів
- **Тестування знань** — створення тестів з автоперевіркою, лімітом спроб і часу
- **Видача сертифікатів** — автоматично при проходженні тесту, PDF-генерація у браузері
- **Оцінювання курсів** — рейтинг 1-5 зірок + коментарі від слухачів
- **Управління користувачами** — адмін змінює ролі, бачить всіх юзерів
- **Захист даних** — 6 шарів безпеки від зовнішнього світу до бази даних
- **Експорт результатів** — CSV-звіти для викладачів і адмінів

Готовність до масштабування

Що	Поточний стан	Що потрібно для масштабу
Файли	Локальний диск (ефемерний)	S3 / Cloudinary через <code>django-storages</code>
Кеш	<code>LocMemCache</code> (в пам'яті процесу)	Redis для розподіленого кешу
База даних	Supabase Free (500 MB)	Supabase Pro або RDS для навантажень
Backend	Один gunicorn-контейнер	Горизонтальне масштабування (JWT stateless)
Frontend	Vercel CDN (глобальний)	Вже масштабований через Edge Network